

HOMEWORK

线性回归模型课后作业解答

Python Scikit-Learn 实践与代码推导

Q1: 为什么 X 必须是列向量?

scikit-learn 的数据格式要求

在 scikit-learn 中, 模型 (如 LinearRegression) 的 fit(X, y) 方法严格要求特征矩阵 **X 必须是二维数组**, 即形状为 (n_samples, n_features)。

变成行向量会怎样?

如果传入的是一维的行向量 (如形状为 (6,) 的数组), 算法会将其误认为只有 1 个样本、包含了 6 个特征。这不仅与实际含义不符, 还会直接引发 **ValueError** 报错:

"Expected 2D array, got 1D array instead."

正确做法:

使用 **x.reshape(-1, 1)** 将一维数组转换为 n 行 1 列的二维列向量。

代码演示与对比

```
# 错误示范: 一维行向量
x = np.array([5, 15, 25, 35])
model.fit(x, y) # -> ValueError!
```

```
# 正确做法: 转换为二维列向量
x = x.reshape(-1, 1)
# x 变为:
# [[ 5]
# [15]
# [25]
# [35]]
model.fit(x, y) # -> 训练成功
```

Q2: 如何显示生成的“熟模型”?

$$y = b0 + b1 * x$$

模型训练完成后，核心参数被保存在模型对象的特殊属性中：

获取截距 b0 (Intercept)

调用代码：

```
b0 = model.intercept_
```

截距是一个标量，代表回归直线在 Y 轴上的截距。示例数据中的结果：

```
b0 = 5.6333
```

获取斜率 b1 (Coefficient)

调用代码：

```
b1 = model.coef_[0]
```

斜率通常是一个数组（因为可能有多个自变量），对于一元线性回归，取第一个元素即可。示例结果：

```
b1 = 0.5400
```

Q3: 如何评价结果的好坏? (R^2 指标)

在 Scikit-Learn 中, 直接使用 `model.score(x, y)` 方法来获取模型的判定系数 (R^2)。

R^2 的统计学含义

决定系数 (Coefficient of Determination) 表示因变量的方差中有多少比例可以由自变量解释。

- **取值范围:** 通常在 0 到 1 之间。
- **判别标准:** R^2 越接近 1, 表示模型拟合度越好, 观测点越靠近拟合直线; 越接近 0 则表示解释能力越差。
- **公式:**
$$R^2 = 1 - \frac{SSE}{SST}$$

示例代码的评估结果

对于博客中提供的数据集:

`x = [5, 15, 25, 35, 45, 55]`

`y = [5, 20, 14, 32, 22, 38]`

执行 `model.score(x, y)` 的结果为:

$R^2 \approx 0.7159$

说明 x 解释了 y 约 71.6% 的方差变化, 模型拟合良好。

Q4: 调整训练与测试比例对模型的影响

通过 `train_test_split` 划分不同的数据集比例。通常情况下，**训练数据越多，模型学习越充分，理论准确度越高**；但若测试集过小，则模型在测试集上的表现 (R^2) 可能因为随机波动而变得不稳定。

代码实验设置

我们生成 200 个带噪声的随机数据点，设定真实的线性关系，并测试不同 `train_size` 下模型在测试集上的 R^2 分数：

```
from sklearn.model_selection  
import train_test_split
```

```
X_train, X_test, y_train, y_test =  
    train_test_split(X, y, train_size=ratio)
```

实验结果对比

- 训练集 **10%** (测试集 90%) -> 测试 $R^2 \approx 0.9712$
- 训练集 **30%** (测试集 70%) -> 测试 $R^2 \approx 0.9703$
- 训练集 **50%** (测试集 50%) -> 测试 $R^2 \approx 0.9695$
- 训练集 **70%** (测试集 30%) -> 测试 $R^2 \approx 0.9687$
- 训练集 **90%** (测试集 10%) -> 测试 $R^2 \approx 0.9744$

结论：在简单线性问题中，少量的样本即可拟合出较好模型；但在复杂任务中，过小的训练集往往会导致模型欠拟合，而过小的测试集则不能稳定反映真实泛化能力。

Q5: 最小二乘法的原理与纯代码实现

数学公式推导

基于误差平方和最小化，一元线性回归的系数可通过以下公式直接求解：

1. 斜率 b_1 :

$$b_1 = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}$$

2. 截距 b_0 :

$$b_0 = \bar{y} - b_1 \bar{x}$$

Python 代码实现 (Numpy)

```
x_mean = np.mean(x)
y_mean = np.mean(y)

# 计算分子 (协方差) 和分母 (方差)
numerator = np.sum((x - x_mean) * (y - y_mean))
denominator = np.sum((x - x_mean)**2)

# 计算斜率和截距
b1_ls = numerator / denominator
b0_ls = y_mean - b1_ls * x_mean

# 打印结果
print(f"y = {b0_ls:.4f} + {b1_ls:.4f} * x")
# 输出: y = 5.6333 + 0.5400 * x
# 结果与 scikit-learn 完全一致!
```

解答完毕

感谢您的查阅与指正